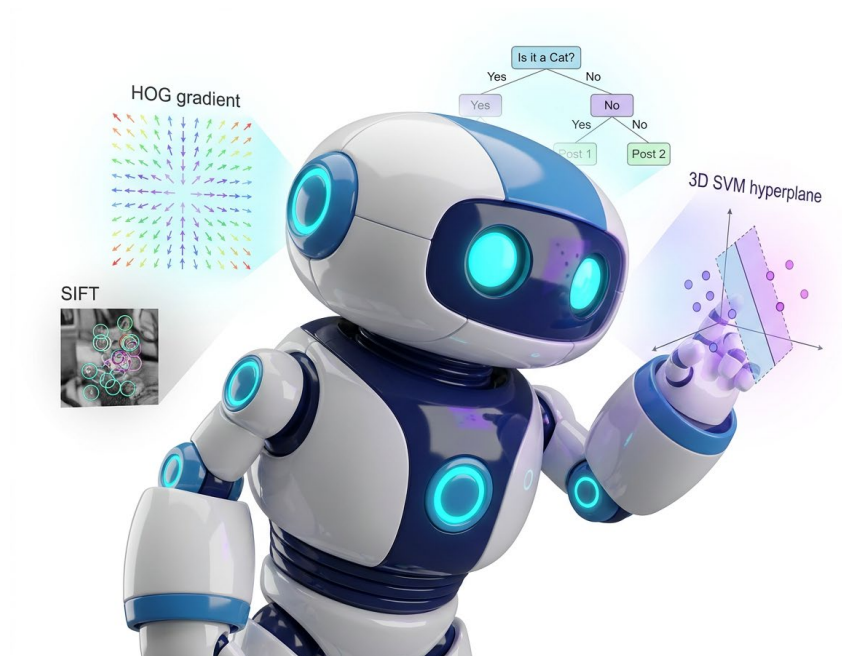


ITAI 1378

Computer Vision for Artificial Intelligence

Module 03

Classic Machine Learning for Computer Vision



Hand crafted features, statistical classifiers, and the era that built the foundation of modern AI.

Author: Patricia McManus
Collaborator: Claude (Anthropic)

Welcome to Module 03

Where We Are

In Module 02 you learned what a pixel really is, how images are stored, and the operations that prepare raw camera data for any kind of analysis. By the end of that module you could take any image and clean it, enhance it, and standardize it. You knew which color space to pick, which file format to use, and which preprocessing operation to reach for in different situations.

Module 03 picks up where preprocessing ends. You have clean, standardized images. Now what? How does a computer actually decide whether a photo contains a cat, or detect a defect on a circuit board, or count the people in a crowd? Before deep learning took over, the answer was a two part recipe: humans designed features, and machines learned to classify them. That recipe ran the entire industry for forty years and still ships in production systems today.

What You Will Learn

By the end of this module you will be able to:

- Explain why classic machine learning still ships in production, in medical imaging, automotive, embedded vision, and regulated industries.
- Walk through the four stages of the classic CV pipeline (preprocess, extract features, classify, predict) and explain what happens at each stage.
- Describe the three feature families (edges and gradients, keypoints, textures) and name the main techniques in each family.
- Pick the right classifier (SVM, Random Forest, k Nearest Neighbors, Naive Bayes, Decision Tree) for a given problem and explain why.
- Tell the difference between supervised and unsupervised learning and recognize where each one is used.
- Map any real CV project to the five stage machine learning lifecycle from data collection to deployment.
- Decide when classic ML is the right tool versus when deep learning is.

Why Should You Care

Three reasons. First, classic ML is the vocabulary of the field. Words like feature, classifier, training set, accuracy, supervised learning, and overfitting were invented during this era. Modern deep learning inherits all of them. Without the foundation, the modern stuff makes less sense and your interviews go worse.

Second, classic ML is still in production. Open up any FDA approved medical imaging device, any automotive driver assistance system, any factory inspection rig, and there is a good chance the actual decision is being made by an SVM or a Random Forest, not a neural network. These methods are interpretable, fast, and work with small datasets. In many industries they win on practical grounds.

Third, every CV engineer needs to be able to choose the right tool for the job. A senior engineer who reaches for deep learning every time will be wrong about thirty percent of the time. The students who graduate able to make that decision are the ones who get the job offers.

Part 01 | Why Classic ML Still Matters

There is a common assumption among new students that classic machine learning is obsolete, replaced by deep learning the way digital cameras replaced film. That assumption is wrong, and it causes new engineers to skip material that they need. This part of the module explains why classic ML is still essential, both as foundation and as a daily working tool.

A Short History

Computer vision as a research field began in the 1960s, with MIT's Summer Vision Project of 1966 being the legendary starting point. The project's plan was to solve vision in a summer. Sixty years later we are still working on it. For the first forty years, computer vision was built on hand crafted features and statistical pattern recognition. Engineers wrote code that detected edges, found corners, measured textures, and compared shapes. Then they fed those numbers into a classifier (a mathematical model that learned to map numbers to labels) and called the result a vision system.

The arrival of deep learning around 2012, with the AlexNet paper, changed the picture. Instead of humans designing features, the network learned them from raw pixels. The change was dramatic. On the ImageNet benchmark, accuracy jumped overnight from around seventy four percent to around eighty four percent, and it has been climbing ever since. Within a few years, deep learning dominated academic research and most consumer products.

But academic dominance is not the whole story. Many production systems, especially those that must be explainable, fast, embedded, or trained on small datasets, kept right on using classic methods. Some of those systems are running on your phone, in your car, in your local hospital, and in the factories that make your food. They never went anywhere.

Where Classic ML Lives Today

Four broad domains rely heavily on classic ML for vision, and you will run into them in your career.

Medical Imaging

Hospitals and medical device companies build CV systems that have to be explainable. A radiologist needs to know not just that a lesion was flagged but why. Regulatory agencies like the FDA require traceable evidence that the system works correctly and predictably. Classic ML methods, especially Support Vector Machines and Random Forests, fit this need beautifully. The features (intensity, shape, texture) are human readable. The classifier is interpretable. The whole pipeline can be audited.

Embedded and Edge Devices

Industrial cameras, agricultural sensors, drones, retail kiosks, and security systems often run on small processors with no GPU and no internet connection. A deep learning model that needs four gigabytes of memory and a discrete GPU will not run there. A classic ML model with a feature extractor and an SVM can run on a microcontroller and answer in milliseconds. As more devices ship with cameras and intelligence, this niche is growing, not shrinking.

Small Data Problems

Deep learning needs lots of data. Tens of thousands of examples minimum, often millions. Many real industrial problems have a few hundred examples at most. A factory line inspecting a specific custom part will not have a million examples to train on. A medical device for a rare condition will not have a million scans. In small data regimes, classic ML often beats deep learning, sometimes by a wide margin. Hand crafted features encode human expertise, which substitutes for missing data.

Hybrid Pipelines

Many modern CV systems are hybrids. A neural network might detect candidate regions in an image, then a classic ML stage classifies each region. Or a classic feature extractor reduces a giant image to a manageable summary before a network takes over. The senior engineer knows how to mix and match. The junior engineer who only knows deep learning misses these opportunities.



The Vocabulary You Need

Before we dive into pipelines and classifiers, you need a small core vocabulary. These terms appear on every job description in the field and on every interview question. Memorize them. Use them in your conversations. They are the price of admission.

Feature

A measurable property of an image (or of any data) that captures something useful about its content. A feature is usually a number, or a list of numbers. Brightness, number of edges, dominant color, and texture pattern are all features. The whole point of classic ML is to compute features that capture what matters and then use them for decisions.

Feature Vector

The list of features that describes one image, packaged into a single mathematical object. If you compute ten features from an image, the feature vector has ten numbers in it. Every image in your dataset becomes one feature vector. The classifier never sees the image; it sees the feature vector.

Classifier

A function that takes a feature vector as input and produces a label as output. Cat or dog. Pass or fail. Tumor or healthy. The classifier is learned from training data. After training, it can be applied to new inputs.

Training Set, Validation Set, Test Set

The three pools of data every ML project needs. The training set is what the algorithm learns from. The validation set is what you use during development to tune choices and avoid overfitting. The test set is locked away until the very end and used only once to measure final performance. Mixing these up is one of the most common beginner errors and costs companies real money.

Overfitting

The condition where your model has memorized the training data instead of learning general patterns. Symptoms: great accuracy on training data, poor accuracy on new data. Almost every ML problem has an overfitting risk, and recognizing it early is a senior level skill.

KEY TAKEAWAY

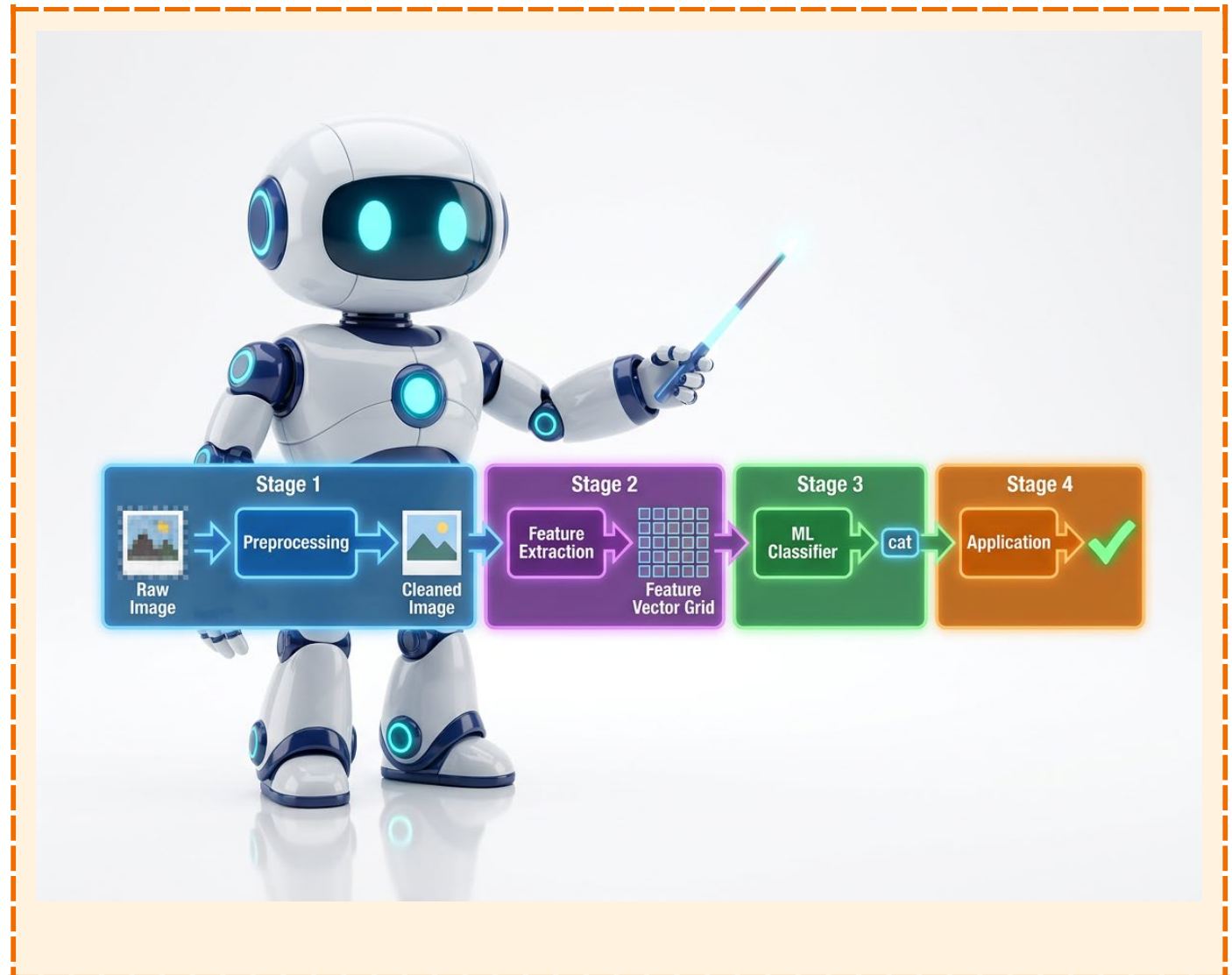
Classic ML is not history. It is the foundation of the field, and it is still in production wherever interpretability, small data, or limited compute matter. Every CV engineer needs to be fluent in this vocabulary.

CAREER CONNECTION | How this shows up in interviews

A typical interview question: 'Describe a CV project where you would choose classic ML over deep learning, and explain why.' Strong candidates answer with a specific scenario (small medical dataset, edge device, regulated industry) and name a specific method (SVM with HOG features, Random Forest with texture features). Weak candidates default to deep learning and cannot explain the trade off.

Part 02 | The Classic CV Pipeline

Every classic CV system follows the same four stage pattern. Once you internalize this pattern you can read a paper, look at a production system, or design your own, and instantly see what each piece is doing. The four stages are: preprocess the image, extract features from it, classify those features, and act on the result.



Stage 1: Preprocessing

Preprocessing is everything you do to the image before you try to extract any meaning from it. Everything in Module 02. The goal is consistency. By the time the image enters stage two, every image in your pipeline should look comparable in size, brightness, contrast, color, and orientation. Without that consistency, the next stages produce garbage.

Four preprocessing operations come up over and over in classic CV. Noise reduction removes random pixel level junk that the sensor introduced. Gaussian blur, median filter, and bilateral filter are the standard tools. Contrast enhancement makes important features more visible by stretching the brightness range. Histogram equalization and gamma correction are the most common methods. Normalization standardizes the pixel value range, typically scaling everything to between zero and one. Geometric correction fixes tilts, distortions, and alignment issues using rotation, scaling, and perspective transform.

Industry rule of thumb

Half of the work on a classic CV system is preprocessing. If your classifier is performing poorly, the first place to look is not the classifier. It is the input.

Stage 2: Feature Extraction

This is the heart of classic ML. Feature extraction is the step where humans encode their understanding of what matters in an image. You write code that says: 'Compute the brightness here, count the edges there, measure how much red is in this region, find the corners over here.' The output is a feature vector, a fixed length list of numbers.

The reason this stage matters so much is that the classifier in stage three only ever sees the feature vector. It never sees the original image. So if your feature extractor throws away the information that matters, no amount of classifier wizardry can recover it. A good feature extractor captures everything important; a bad one loses the signal and keeps the noise.

Part 03 of this booklet covers feature extraction in depth. For now, just know that the three families of techniques are edge and gradient features, keypoint features, and texture features. Each family captures a different aspect of what is in the image.

Stage 3: Classification

The classifier is the only learnable part of a classic CV pipeline. Everything before it (preprocessing, feature extraction) is designed and coded by humans. The classifier takes the feature vectors and the

corresponding labels from your training set, learns a function that maps vectors to labels, and then applies that function to new vectors.

Different classifier types make different assumptions about how features relate to labels. A Support Vector Machine assumes the classes can be separated by a hyperplane in feature space. A Decision Tree assumes the answer can be found by asking a sequence of yes or no questions. A k Nearest Neighbors classifier assumes that similar feature vectors should have similar labels. None of these assumptions is universally right. Picking the classifier is a real engineering decision, covered in Part 04.

Stage 4: Prediction and Action

The fourth stage is what the system does with the classification. In a quality inspection rig, the action might be 'reject the part and stop the line'. In a medical screening tool, the action might be 'highlight the region for the radiologist to review'. In a self driving prototype, the action might be 'apply the brakes'. In a research notebook, the action might be 'write the prediction to a results file'.

This stage is the easiest to underestimate. New engineers focus on accuracy numbers and forget that the system has to do something. A pipeline that gets ninety nine percent accuracy on a benchmark but produces a result an hour after the part has passed the inspection station is useless. The action stage drives requirements all the way back through the pipeline.

INDUSTRY PRACTICE | Where the human ends and the machine begins

Notice that humans design stages one, two, and four. The machine only learns stage three. This is the fundamental difference between classic ML and deep learning. In deep learning, the machine learns all four stages at once from raw pixels to final output. That shift, from humans designing features to machines learning them, is the single biggest leap in modern CV.

Walking Through a Concrete Example

Imagine a factory line inspecting circuit boards. The goal is to detect missing solder joints. Here is what the four stages might look like in practice.

Stage 1, preprocessing: the camera captures the board. The image is converted to grayscale (color is not relevant for this problem). The image is denoised with a small Gaussian blur to remove sensor noise. The image is normalized so that lighting differences across boards do not affect the result.

Stage 2, feature extraction: a region around each expected solder joint is cropped out. For each region, several features are computed: average brightness, edge density, and a local binary pattern descriptor of the texture. The result for each joint is a feature vector with maybe ten numbers in it.

Stage 3, classification: a Support Vector Machine has been trained on thousands of feature vectors, half from good joints and half from missing or defective joints. Given a new feature vector, the SVM predicts whether the joint is good or bad.

Stage 4, action: if any joint is predicted bad, the system raises an alert and the board is routed to a human inspector. Otherwise the board moves on. A log entry is written for every board.

That entire pipeline can run on a small industrial PC, makes decisions in milliseconds, can be explained to a quality engineer, and can be updated by adding new training examples. It is the kind of system that ships in production today and pays the bills for many CV engineers.

KEY TAKEAWAY

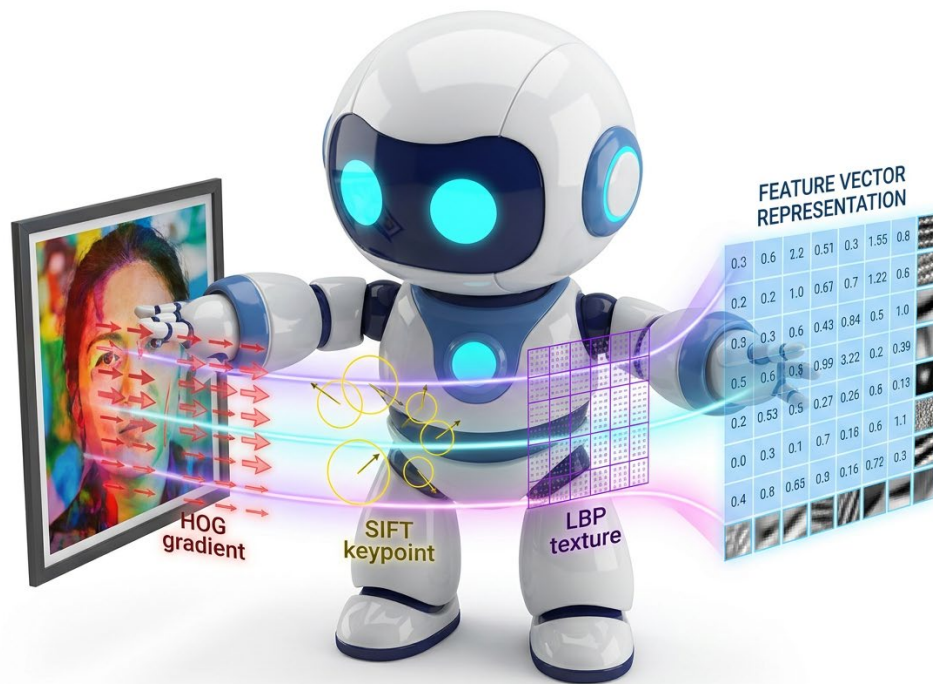
The classic CV pipeline has four stages: preprocess, extract features, classify, act. Stages one, two, and four are designed by humans. Only stage three is learned. Understanding this boundary is the single most important concept in this module.

KNOWLEDGE CHECK | Quick check on the pipeline

1. A factory inspection system fails on shiny parts under bright lights but works fine on matte parts. Which pipeline stage should you investigate first, and why?
2. Name the only stage of the classic CV pipeline that the machine learns from data. What gets learned at that stage?
3. A coworker claims their CV system has 'no preprocessing'. What are they probably doing without realizing it?

MISCONCEPTION ALERT | Accuracy is not the only number that matters

Beginner engineers report a single accuracy number and call the system done. Senior engineers report at least three: accuracy, latency, and cost per inference. A pipeline that hits 99 percent accuracy but takes 30 seconds per image is useless on a factory line moving parts every two seconds. Always frame performance as a triple, not a single number.



Part 03 | Feature Engineering

Feature engineering is the part of classic ML where engineers earn their reputations. The choice of features makes or breaks a pipeline. Two engineers facing the same problem and the same dataset can produce results that differ by twenty accuracy points purely based on which features they decide to compute. This part teaches you the three families of features and the main techniques in each.

What Is a Feature, Really

A feature is a number, or a list of numbers, that summarizes something useful about an image. Useful in what sense? In the sense that the feature helps the classifier tell different classes apart. If you are classifying cats and dogs, a useful feature would distinguish cat ears from dog ears. A useless feature would measure something irrelevant, like the average brightness of the background sky.

Good features have three properties. They are discriminative (different classes produce different feature values). They are robust (small changes in lighting, position, or scale do not change the feature values much). They are computable (you can write reasonably fast code to compute them). Designing features that satisfy all three is hard, which is why feature engineering has historically been considered an art.

💡 THINK ABOUT IT | Why does it matter that the classifier never sees the original image?

If your features throw away information, the classifier cannot recover it. Pick features that capture what matters for your specific problem. There is no universal feature set; the right choice depends on what you are trying to detect.

Family 1: Edge and Gradient Features

Edges are places in an image where the brightness changes rapidly. The boundary between a dark object and a light background is an edge. So is the line where one color meets another. Edge based features capture shape information.

Sobel and Canny Edge Detection

The Sobel filter is the standard tool for detecting edges. It computes the brightness gradient at each pixel: how fast is the brightness changing, and in which direction. The Canny edge detector is a more refined version that produces clean, thin edges by combining Sobel with smart thresholding.

Edges alone are not features yet; they are pixel locations. To turn them into features, you typically count them, measure their orientations, or compute statistics about their layout. Five hundred horizontal edges with strong contrast is a different signature from twenty diagonal edges with weak contrast.

HOG: Histogram of Oriented Gradients

HOG is the most famous edge based feature in classic CV. It was developed in 2005 and powered the first reliable pedestrian detection systems. The idea is simple. Divide the image into small cells. In each cell, count how many edges point in each direction (up, down, left, right, and various diagonals). The result for the whole image is a histogram, a list of counts, that captures the dominant edge orientations everywhere.

HOG works well for objects with distinctive shapes: pedestrians, vehicles, faces. It is invariant to brightness changes (because it counts gradients, not raw pixels) and reasonably robust to small position changes. For a decade, HOG plus a Support Vector Machine was the best general purpose object detection pipeline available.

Haar Features

Haar features compute simple brightness differences between rectangular regions. Are the pixels in this rectangle brighter than the pixels in that adjacent rectangle? The famous Viola Jones face detector from 2001 used thousands of Haar features and ran in real time on a webcam. Haar features are extremely fast to compute, which is why they powered the first practical face detection in digital cameras.

Family 2: Keypoint Features

Keypoint features focus on distinctive landmarks in an image rather than overall shape. The corner of a building. The eye in a portrait. The intersection of two lines. These landmarks can be matched between images, which is useful for tasks like image stitching, object recognition, and tracking.

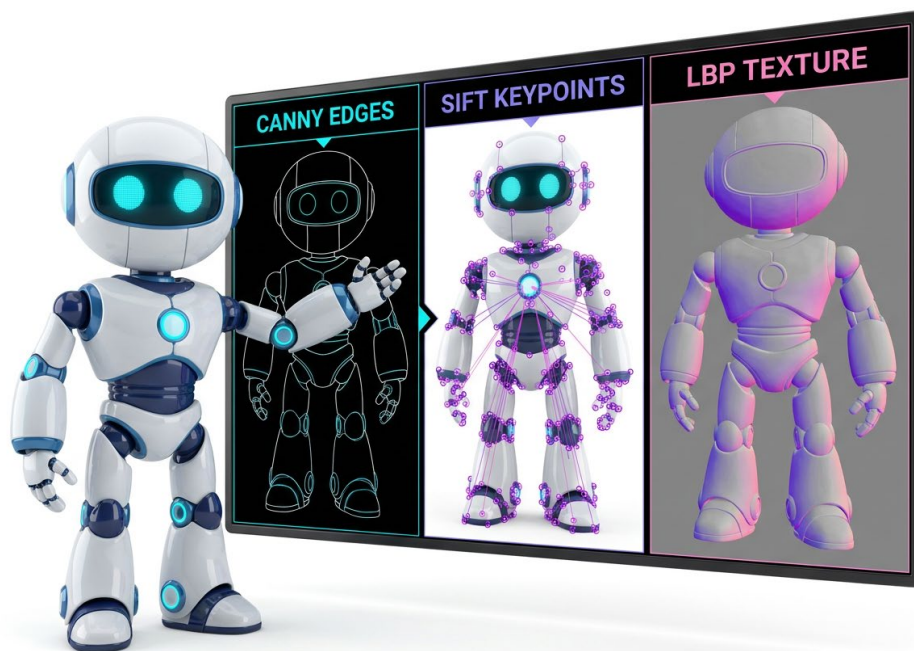
SIFT: Scale Invariant Feature Transform

SIFT was the dominant keypoint method for fifteen years. It finds distinctive points in an image and computes a rich descriptor for each one. Two key properties make SIFT special. It is scale invariant (the same point will be found whether the image is large or small). It is rotation invariant (the same point will be found whether the image is upright or sideways).

SIFT powered the early panorama stitching in cameras, the first augmented reality apps, and many object recognition systems. Its main downsides are computational cost and patent licensing (the patent has now expired). SURF and ORB are faster open source alternatives that solve similar problems.

Corner Detectors

The Harris corner detector and the FAST corner detector are lighter weight keypoint finders. They locate corners (places where the image has strong gradients in multiple directions) but do not produce as descriptive a fingerprint as SIFT. They are useful when you need many landmarks fast and can match them by other means, such as in real time tracking on mobile devices.



Family 3: Texture Features

Texture features capture the repeating or stochastic patterns on surfaces. The bark of a tree. The weave of fabric. The cellular structure visible in a microscope image. Texture is information that is not captured by edges or corners; it lives in the statistical relationships between nearby pixels.

LBP: Local Binary Patterns

LBP is the simplest texture feature and one of the most useful. For each pixel, you compare it to its eight neighbors. If the neighbor is brighter, write a one; if darker, write a zero. The result is an eight bit pattern that describes the local texture. Compute LBP for every pixel and you get a texture map of the whole image. A histogram of LBP patterns is a compact texture descriptor.

LBP is extremely fast, robust to lighting changes, and produces excellent results for face recognition, material classification, and medical texture analysis. It is one of the workhorses of embedded computer vision.

Gabor Filters

Gabor filters detect specific spatial frequencies and orientations. They are inspired by the human visual cortex (where neurons respond to similar patterns). A bank of Gabor filters tuned to different scales and orientations produces a rich multi scale texture descriptor. Gabor features are still used in iris recognition systems and in some medical imaging applications.

GLCM: Gray Level Co-occurrence Matrix

GLCM computes how often pairs of pixel values appear at specific offsets in an image. From the resulting matrix, you can derive measures of contrast, homogeneity, and energy. GLCM features are workhorses of remote sensing (analyzing satellite imagery) and texture classification in materials science.

INDUSTRY PRACTICE | Picking features for your problem

If your problem involves shape and outline (cars, pedestrians, faces, parts on a conveyor), start with edge based features (HOG). If your problem involves matching specific points or tracking (image stitching, object tracking), use keypoint features (SIFT, ORB). If your problem involves surface patterns or material properties (skin lesions, fabric defects, food quality), use texture features (LBP, GLCM).

⚠ MISCONCEPTION ALERT | You do not need to know the math

A common misconception is that working with classic features requires deep math. It does not. Modern libraries (OpenCV, scikit image) wrap every feature in a one line function call. You compute HOG with one line, SIFT with one line, LBP with one line. The skill is knowing what to compute, not how to compute it from scratch.

📊 MATH MOMENT (optional) | What is a feature vector, really

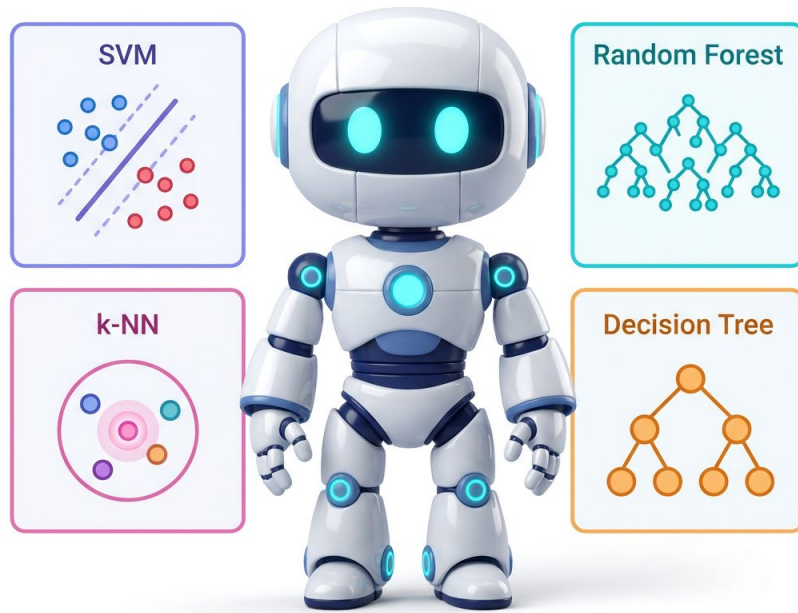
Think of a feature vector as a row in a spreadsheet. Each column is one feature (brightness, edge count, dominant color, and so on). Each row is one image. A dataset of 1,000 labeled images becomes a spreadsheet with 1,000 rows and however many columns you computed. The classifier looks at this spreadsheet, learns which columns predict which labels, and saves the rule. No deeper math required to use it; the libraries handle the arithmetic.

✅ KNOWLEDGE CHECK | Quick check on features

1. You are detecting defective bottles on a conveyor belt. Bottles tilt slightly. Which feature family handles tilt invariance best, and why?
2. An engineer says they will use HOG features to classify the texture of leather samples. What is wrong with that choice?
3. Why does the classifier work with the feature vector instead of the original pixels?

Part 04 | The Workhorse Classifiers

Once you have feature vectors, you need a classifier. This part introduces the five classifiers that handle the vast majority of classic CV work in production. None of them is universally best. Each makes different assumptions about how features relate to labels, and the right choice depends on your data, your problem, and your constraints.



Support Vector Machine (SVM)

The SVM is the most famous classifier in classic ML for vision. The idea is geometric. Imagine your feature vectors plotted as dots in space, with cat dots on one side and dog dots on the other. The SVM finds the best straight line (or curved surface, in higher dimensions) that separates the two groups, with the widest possible gap on either side.

SVMs are particularly strong on small to medium datasets, on high dimensional feature spaces (where HOG and SIFT live), and on problems where the classes are well separated. They are mathematically elegant, well studied, and ship in countless production systems. Their main weakness is scaling: an SVM on a million examples becomes slow to train and slow to use.

When to use SVM

Reach for SVM when your training set has fewer than ten thousand examples, when your features are high dimensional (more than a hundred dimensions), and when interpretability is less important than predictive accuracy. SVMs paired with HOG features ran most of the world's pedestrian detection systems for ten years.

Random Forest

A Random Forest is an ensemble. Instead of one classifier, you train hundreds of decision trees, each on a slightly different subset of the data, and let them vote on the answer. The voting averages out the mistakes of individual trees and produces robust predictions.

Random Forests are the easiest classifier to make work well. They handle mixed feature types (numerical, categorical, boolean) without preprocessing. They tolerate noise in the data. They produce a built in measure of feature importance, telling you which features mattered most for the predictions. And they almost always perform reasonably well even without tuning.

When to use Random Forest

Reach for Random Forest when you want a strong baseline fast, when your features are heterogeneous, when you need interpretability (the feature importances), and when you do not have time to tune hyperparameters carefully. In industry settings where a project must produce a working system in two weeks, Random Forest is often the first tool out of the box.

k Nearest Neighbors (k-NN)

The k-NN classifier is the simplest of all. To classify a new feature vector, find the k training examples that are most similar to it (closest in feature space) and let them vote. That is the entire algorithm. There is no training step. The model is just the training set itself.

k-NN is sometimes called lazy learning because it does no work until prediction time. Its strengths are simplicity and no training cost. Its weaknesses are slow prediction (you have to scan all training data for each new input) and sensitivity to the choice of distance function and the value of k.

When to use k-NN

Reach for k-NN when you need a quick baseline, when your dataset is small enough that scanning it for each prediction is cheap, or when you have no training time available. A reasonable rule of thumb: if a Random Forest is the easiest classifier to make work, k-NN is the easiest classifier to understand and implement from scratch.

Naive Bayes

Naive Bayes is a probabilistic classifier. It treats every feature as independent evidence for or against each class, computes the probability of each class given the evidence, and picks the most probable class. The 'naive' part is the independence assumption: in real data, features are not actually independent, but the assumption simplifies the math so much that the algorithm runs fast and often works well anyway.

Naive Bayes is famously good at text classification (spam detection, document categorization) but also has uses in vision when speed matters and the features are roughly independent. Its main weakness is that it gives only crude probability estimates when the independence assumption is badly violated.

When to use Naive Bayes

Reach for Naive Bayes when prediction speed is the top priority, when your features are reasonably independent, or when you want a transparent probabilistic answer (this image is seventy three percent cat, twenty one percent dog, six percent bird). It is also a strong choice for problems where you have very little training data.

Decision Tree

A single Decision Tree builds a flowchart of yes or no questions. Is the brightness above one hundred and twenty? If yes, go left; if no, go right. Is there an edge in the top half? And so on. At the leaves of the tree, the answer is given. Decision trees are the most interpretable classifier in classic ML; you can print the tree and follow the reasoning.

On their own, single trees are not as accurate as Random Forests (which average many trees). But for problems where you must explain every decision, a single tree often beats more accurate but opaque alternatives. In medical and legal applications, this is a real advantage.

When to use a single Decision Tree

Reach for a Decision Tree when interpretability is the dominant requirement, when you need to defend each decision in regulatory or legal terms, or when you want a quick check on whether a problem is even learnable.

CAREER CONNECTION | Choosing a classifier in five steps

When facing a new classification problem, work through this checklist:

1. Are you trying to win a benchmark or ship to production? Production usually prefers Random Forest or SVM.
2. How much training data do you have? Less than a few hundred examples: Naive Bayes or k-NN. Few thousand: SVM. Tens of thousands: Random Forest. Hundreds of thousands or more: consider deep learning.
3. Do you need to explain every decision? Decision Tree first, then Random Forest with feature importances.
4. How fast must prediction be? Fastest is Naive Bayes or k-NN with a small dataset. SVM and Random Forest are also fast at prediction time.
5. What is the cost of being wrong? In life critical settings, run multiple classifiers and require agreement. In low stakes settings, pick the easiest to train.

A Decision Table

Use the following table as a quick reference when starting a new project. The right classifier depends on the constraints of your problem, not on which classifier is academically fashionable this year.

Less than 1000 samples: SVM with RBF kernel is the typical first choice. It handles small data well and works in high dimensional feature spaces.

Need to explain decisions: Decision Tree or Random Forest. The tree structure is auditable.

Real time inference required: Naive Bayes or small k-NN. Both classify in microseconds.

Mixed feature types (numerical, categorical, boolean): Random Forest handles them natively without preprocessing.

Quick baseline, no training time: k-NN. Zero training required.

Lots of data, complex perceptual problem: at this point you have probably outgrown classic ML. Consider deep learning.

INDUSTRY PRACTICE | The real engineering rule

Try two or three classifiers and compare. Do not commit to one before testing. Modern libraries like scikit learn let you swap classifiers with a single line of code. The five minutes you spend running a quick comparison will save you hours of regret later.

MATH MOMENT (optional) | The SVM in one mental picture

Imagine your feature vectors as dots on a piece of paper. Cats are blue dots, dogs are red dots. The SVM draws the best straight line between them with the widest possible gap (the margin) on either side. In higher dimensions the line becomes a curved surface, but the idea is the same. Picking a new point and asking 'which side of the line is it on?' becomes the prediction. No equations needed to use it; libraries handle the geometry.

THINK ABOUT IT | Why try several classifiers instead of picking one?

Different classifiers make different assumptions about how features relate to labels. SVM assumes the classes can be separated cleanly. Random Forest assumes the answer can be reached by a sequence of yes or no questions. k-NN assumes similar things have similar labels. The data does not announce which assumption is closest to true. Running two or three lets the data tell you.

KNOWLEDGE CHECK | Quick check on classifiers

1. You have 250 labeled images of healthy and diseased plant leaves. Which classifier should you try first, and why?
2. Your team needs to explain every decision to FDA reviewers. Which classifier is the easiest to defend, and why?
3. Name one situation where k-NN beats Random Forest in practice.

Part 05 | Supervised, Unsupervised, and the ML Lifecycle

So far this booklet has talked about classifiers that learn from labeled examples. That is one flavor of machine learning called supervised learning. There are two others. Understanding the three flavors of ML, plus the lifecycle of every real project, is what separates students who can talk about ML from students who can actually build it.

Supervised Learning

Supervised learning is the default flavor and what every classifier in Part 04 used. You collect training data where every example has a label (cat or dog, defect or good, malignant or benign). The algorithm learns to map inputs to labels. After training, you can predict labels for new inputs.

Most CV jobs in industry are supervised. The labels can be expensive (a radiologist has to mark every scan), tedious (someone has to draw bounding boxes on a million images), or sometimes both. Companies that do CV well have either spent heavily on labeled data or invented clever ways to get labels cheaply (active learning, crowdsourcing, weak supervision).

Unsupervised Learning

Unsupervised learning has no labels. You give the algorithm a pile of feature vectors and ask it to find patterns. The two main tasks are clustering (group similar items together) and dimensionality reduction (find a simpler representation of the data).

In CV, unsupervised methods show up in three places. Organizing photo libraries (group photos that look alike, like vacation photos versus office photos). Detecting anomalies (find images that look unlike anything in the normal training set). Pretraining (use unsupervised methods to find general features in unlabeled data, then fine tune with a smaller labeled set).

k-Means Clustering

The most famous clustering algorithm. You decide how many groups you want (say, five). The algorithm finds five center points and assigns every feature vector to its closest center, then updates the centers and repeats until things settle down. The result is k groups of similar items. K-means powers color quantization (reducing an image to its k most common colors), image segmentation (grouping similar pixels), and the organize by similarity features in photo apps.

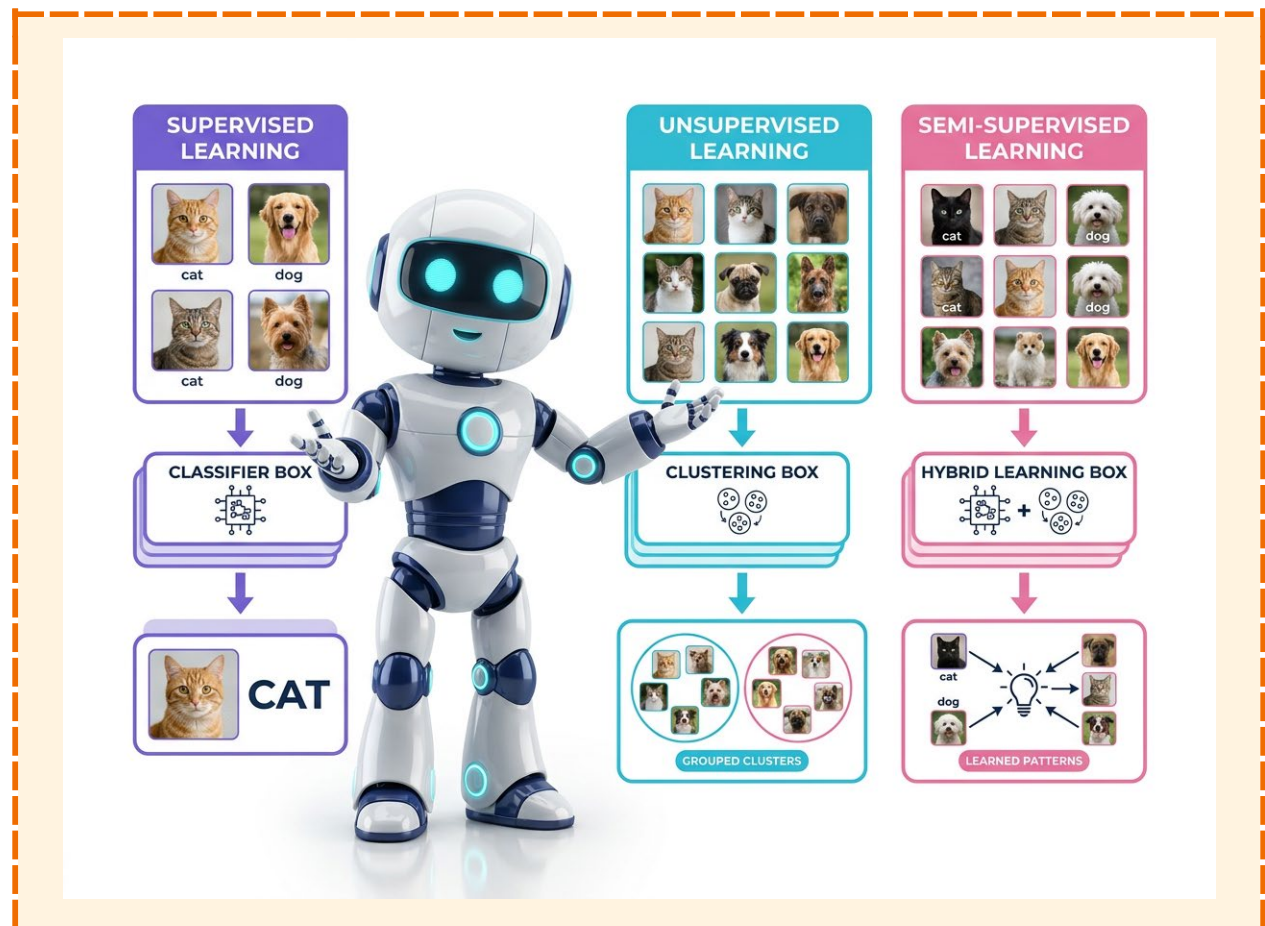
PCA: Principal Component Analysis

PCA is the most famous dimensionality reduction method. Given high dimensional feature vectors (say, hundreds of dimensions), PCA finds a smaller number of new dimensions (say, ten) that capture most of the variability in the data. PCA is used to compress feature vectors, to visualize high dimensional data in two or three dimensions for human inspection, and to clean up noisy data before classification.

Reinforcement Learning

Reinforcement learning is the third flavor, used when the algorithm learns by trial and error in an environment. There is no fixed training set; the algorithm takes actions, gets rewards or penalties, and updates its strategy over time. RL is the foundation of game playing AI (AlphaGo, AlphaStar), robotics, and some of the work on autonomous vehicles.

RL is mostly outside the scope of this course. We mention it so you know it exists and so you can recognize when a colleague is talking about it. For ITAI 1378, supervised learning is the main game, with occasional appearances by unsupervised methods.



The ML Lifecycle

Every real machine learning project, classic or modern, goes through the same five stages. New engineers think of ML as a one shot training step; senior engineers think of it as a loop that runs for the life of the product. Understanding the lifecycle is what gets you from a notebook demo to a system that ships.

Stage 1: Collect Data

You cannot train a model without data. For classic ML on a CV problem, you need images plus labels. How many? It depends. For a simple binary classification with strong features, a few hundred per class can work. For more complex problems, thousands per class. For deep learning, often millions. The collection step is usually the slowest and most expensive part of any ML project.

Stage 2: Prepare and Clean

Raw data is never ready for training. Some images are wrong (misabeled, corrupted, duplicate). Some classes have far more examples than others (class imbalance). Some are of poor quality. You filter, fix, augment, and split into training, validation, and test sets. Quality of preparation directly determines quality of the final model.

Stage 3: Train

This is the headline step, even though it is usually the smallest fraction of total work. You feed the prepared data into the chosen classifier with the chosen features and let it learn. For classic ML methods on reasonable datasets, training takes seconds to minutes. For deep learning, hours to weeks.

Stage 4: Evaluate

Once trained, the model must be evaluated on data it has never seen (the test set). Compute the accuracy. Compute the confusion matrix to see which classes get mistaken for which. Look at examples the model gets wrong; they often reveal data problems or feature problems. Do not stop here; in production, you also evaluate against business metrics like cost per false positive.

Stage 5: Deploy and Monitor

The model goes into a real system. It runs on real inputs, makes real predictions, and affects real users. Performance is monitored continuously. New data comes in. Edge cases are discovered. The model is retrained. The cycle repeats. Models that ship and stay current require ongoing engineering effort, not one shot heroics.

 INDUSTRY PRACTICE | The step beginners skip

Stage 5 is the step that beginners almost always skip and that senior engineers obsess over. Production performance is what matters. A model that gets ninety nine percent accuracy on a test set in a notebook but degrades within a month of deployment because the data has shifted is useless. The ability to monitor, detect drift, and retrain is a senior level skill that takes years to build.

 MATH MOMENT (optional) | Bias and variance in plain English

Bias is how wrong your model is on average. Variance is how much your model's predictions change when you retrain it on slightly different data. High bias means the model is too simple and misses patterns (it gets the wrong answer the same way every time). High variance means the model is too flexible and chases noise (it gets a different answer depending on which data points you happened to include). The art of ML is finding the sweet spot. Reach for a simpler model if you have high variance, a more flexible model if you have high bias.

 MISCONCEPTION ALERT | Test set is not validation set

A common beginner mistake is using the test set to tune the model. Once you peek at the test set and adjust anything based on what you saw, the test set is no longer a fair measure of how your model will perform on new data. The validation set is for tuning. The test set is locked away until the very end and is used exactly once. Confusing the two is one of the most expensive mistakes in industry; it produces models that score great in development and fail in production.

 KNOWLEDGE CHECK | Quick check on the lifecycle

1. Name the three data splits and what each one is used for.
2. A customer's photo organization app needs to group similar photos but has no labels. Which flavor of ML applies, and which algorithm fits?
3. Why does the lifecycle keep looping even after the model is deployed?

Part 06 | When Classic ML Wins (and When It Loses)

This module closes with the engineering question that every CV practitioner has to answer over and over: when do I use classic ML, and when do I reach for deep learning? The honest answer is not 'always deep learning'. It depends on the problem, the data, the constraints, and the stakes. This part gives you a working framework for the decision.

When Classic ML Wins

Small Datasets

Deep learning is data hungry. Classic ML methods, especially SVM and Random Forest, can produce excellent results with a few hundred or a few thousand examples per class. If you are working with a custom industrial part, a rare medical condition, or a niche security application, deep learning may not even be an option for lack of data. Classic methods often win by default.

Interpretability Requirements

Some applications must justify their decisions. A medical device that recommends a biopsy. A loan approval system. A regulatory inspection. In these settings, a transparent Decision Tree or a Random Forest with feature importances is far easier to defend than a deep neural network whose internal reasoning is opaque. Even when deep learning would be more accurate, the legal and regulatory cost of unexplainable decisions may rule it out.

Embedded Deployment

A classic ML pipeline can run on a microcontroller with no GPU and no internet connection. A deep neural network typically cannot, or it requires expensive specialized hardware. As CV moves into agriculture, retail, and infrastructure, where devices are cheap and disconnected, classic ML keeps winning by being deployable where deep learning is not.

Domain Knowledge Is Available

When experts already know what matters in the data (radiologists know which features make a tumor suspicious; metallurgists know which textures indicate fatigue), encoding that knowledge as hand crafted features is faster and more reliable than asking a network to learn it. Domain knowledge plus classic ML often beats domain ignorance plus deep learning, especially in early stages of a project.

Latency Constraints

Some systems must respond in microseconds. A safety system on a high speed manufacturing line. A trigger in a missile defense system. A response in a high frequency trading system. Classic ML inference is usually faster than deep learning inference by an order of magnitude, sometimes more. In hard real time settings, classic methods still rule.

When Classic ML Loses

Large Datasets and Complex Perception

On benchmarks like ImageNet, where there are over a million labeled images and a thousand classes, deep learning wins decisively. The complex patterns that distinguish a thousand fine grained categories of dog, bird, and mushroom are exactly the kind of patterns that hand crafted features struggle to capture and that learned features handle naturally. If you are doing general consumer image recognition, deep learning is the right tool.

Top Tier Accuracy

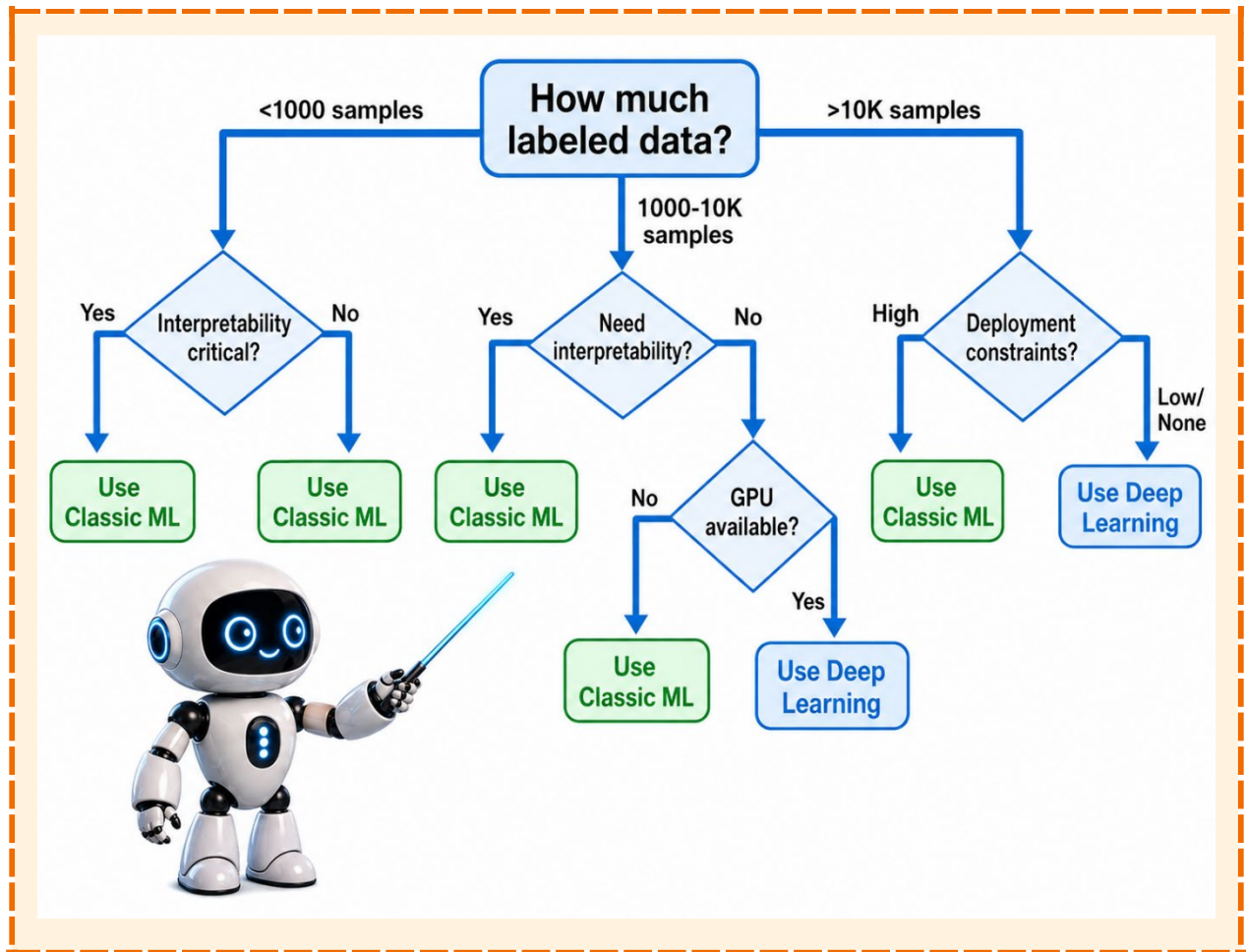
In any modern benchmark, deep learning sets the accuracy ceiling. Classic methods are often a few percentage points behind. When your application needs the absolute best possible accuracy and you have data and compute to back it up, deep learning is the answer.

End to End Learning

Some problems benefit enormously from learning the entire pipeline together rather than designing each stage by hand. Speech recognition was transformed when systems started learning the feature extractor and the classifier jointly. The same happened in vision around 2012 with AlexNet. When the system benefits from joint optimization, deep learning is the path.

Continual Improvement

Deep models can be retrained on new data without throwing away the prior learning. Pretrained models can be fine tuned. Knowledge transfers from one task to another. Classic ML pipelines tend to require redesign of the feature extractor when the problem shifts, which is expensive. If you expect your problem to evolve, deep learning has structural advantages.



The Senior Engineer Perspective

A senior CV engineer does not have a favorite algorithm. They have a toolkit and they reach for the right tool for each problem. They might solve a factory inspection problem with HOG and an SVM in two weeks. They might solve a customer facing image search problem with a pretrained transformer in six months. They know that the right choice depends on the constraints of the application, not on whatever is fashionable this year.

This is the perspective you should develop. Be fluent in classic ML. Be fluent in deep learning. Know when each one is the right answer. Be honest about trade offs. Senior engineers in CV interviews talk this way, and they are the ones who get the offers.

 KEY TAKEAWAY

Classic ML and deep learning are different tools for different jobs. Classic ML wins on small data, interpretability, embedded deployment, and latency. Deep learning wins on large data, complex perception, top tier accuracy, and end to end learning. A senior engineer knows both and picks the right one.

 THINK ABOUT IT | What is your default tool?

A team has a tendency to reach for whatever was cutting edge during their training. If you graduated in 2014, your default was probably SVM. If you graduated in 2024, your default was probably a deep network. Both defaults are wrong; the right default is asking what the problem actually needs. Catch yourself the next time you reach for an algorithm without thinking. Was that the problem talking or your habit?

 KNOWLEDGE CHECK | Quick check on tool selection

1. An automotive supplier needs to detect a specific bracket defect. They have 400 labeled images, must run on an embedded microcontroller, and need to explain decisions to a quality auditor. Which approach, and why?
2. A consumer photo app needs to identify thousands of dog breeds across millions of user uploads. Which approach, and why?
3. Name two trade offs that go against deep learning even when it is more accurate.

Review Questions

Work through these on your own first. Then check your answers against the booklet and your notes. If any question feels uncertain, that is the section to reread.

1. Describe the four stages of the classic CV pipeline. Which stages are designed by humans, and which one is learned by the machine?
2. Name the three families of features in classic ML and give one technique that belongs to each family.
3. Explain in plain language what HOG does and what kind of problem it is best suited for.
4. You have three hundred labeled training images of factory parts. Which classifier should you try first, and why?
5. What is the difference between supervised, unsupervised, and reinforcement learning? Give one concrete CV example of each.
6. List the five stages of the ML lifecycle. Which one do beginners most often skip, and why does it matter?
7. Give two specific scenarios where classic ML beats deep learning, and explain why for each one.
8. What is the difference between a feature and a feature vector? Why does the classifier only see the feature vector and not the original image?
9. Name one CV application where interpretability is more important than maximum accuracy, and explain why.
10. Why does the choice of feature extractor matter more than the choice of classifier in many classic CV systems?

Glossary

Quick reference for the terminology used in this module. If a term comes up in interviews or in your own work and the meaning escapes you, this is the page to bookmark.

Classifier

A function that takes a feature vector as input and produces a label as output. Learned from training data.

Decision Tree

A classifier that learns a flowchart of yes or no questions. Highly interpretable but less accurate than ensembles.

Edge

A location in an image where the brightness changes rapidly. The fundamental building block of shape based features.

Ensemble

A combination of multiple models whose predictions are averaged. Random Forest is the most common ensemble in classic ML.

Feature

A measurable property of an image (or other data) that summarizes something useful. Usually a single number.

Feature Engineering

The work of designing and computing features. The most important step in classic ML and the one that distinguishes good engineers from average ones.

Feature Vector

The list of features that describes one image, treated as a mathematical object. Inputs to a classifier are always feature vectors.

Gabor Filter

A texture feature inspired by the human visual cortex. Detects specific spatial frequencies and orientations.

HOG (Histogram of Oriented Gradients)

An edge based feature that captures the distribution of edge orientations in an image. The basis of classical pedestrian detection.

k Nearest Neighbors (k-NN)

A classifier that predicts based on the labels of the k most similar examples in the training set. No training step.

k-Means

The most common clustering algorithm in unsupervised learning. Groups data into k clusters by repeatedly assigning points to centers and updating the centers.

LBP (Local Binary Patterns)

A simple, fast texture feature based on comparing each pixel to its neighbors. Robust to lighting changes.

Naive Bayes

A probabilistic classifier that treats features as independent evidence. Fast and surprisingly effective.

Overfitting

The condition where a model has memorized the training data and performs poorly on new data. The most common ML pathology.

PCA (Principal Component Analysis)

The most common dimensionality reduction method. Finds new dimensions that capture most of the data's variability.

Random Forest

An ensemble of decision trees, each trained on a slightly different subset of the data. Strong general purpose classifier.

SIFT (Scale Invariant Feature Transform)

A keypoint feature that finds distinctive points and produces rich descriptors. Robust to scale and rotation.

Supervised Learning

Machine learning from labeled data. The most common type in CV jobs.

Support Vector Machine (SVM)

A classifier that finds the optimal separating boundary between classes in feature space. Strong on small to medium datasets.

Test Set

The data held out and used only for final evaluation. Never used during training or development.

Training Set

The data the algorithm learns from. The largest of the three data splits.

Unsupervised Learning

Machine learning from unlabeled data. Used for clustering, dimensionality reduction, and pretraining.

Validation Set

The data used during development to tune choices and check for overfitting. Distinct from the test set.

References and Further Reading

The following resources are excellent next steps for deepening your knowledge of classic ML in computer vision and the modern methods that grew out of it. None of them are required for this course.

Foundational Books

Szeliski, Richard. Computer Vision: Algorithms and Applications, Second Edition. Springer, 2022. A comprehensive textbook covering both classic and modern CV, with chapters on feature detection, classic ML, and modern deep learning. Available free online from the author's website at szeliski.org/Book.

Hastie, Trevor; Tibshirani, Robert; and Friedman, Jerome. The Elements of Statistical Learning, Second Edition. Springer, 2009. The canonical reference for classic ML methods including SVM, Random Forest, and the math behind them. Free PDF available from Stanford.

Forsyth, David and Ponce, Jean. Computer Vision: A Modern Approach, Second Edition. Pearson, 2012. Strong coverage of the classic CV pipeline, features, and statistical methods that this module builds on.

Elgendy, Mohamed. Deep Learning for Vision Systems. Manning Publications, 2020. Bridges the classic CV pipeline (preprocessing, feature extraction, classification) with the modern deep learning approach. Chapter 1 is one of the clearest treatments of the four stage pipeline available, and accessible to readers without a heavy math background.

Recent Books (2024 to 2026)

Ayyadevara, V Kishore and Reddy, Yeshwanth. Modern Computer Vision with PyTorch, Second Edition. Packt Publishing, June 2024. The most current practical CV book on the market. Covers the bridge from classic methods into modern deep learning, with hands on examples of CNNs, transformers, vision transformers, CLIP, Stable Diffusion, and production deployment.

Huang, Howard; Stevens, Eli; Antiga, Luca; and Viehmann, Thomas. Deep Learning with PyTorch, Second Edition. Manning Publications, 2025. Updates the bestselling original with transformers, diffusion models, and generative AI. Strong on fundamentals and on what is current in modern CV.

Chollet, Francois. Deep Learning with Python, Third Edition. Manning Publications, 2025. The accessible introduction by the creator of Keras, fully revised for the modern era. Worked examples in TensorFlow and Keras.

Elgendy, Mohamed. Grokking Deep Learning for Computer Vision. Manning Publications, forthcoming (MEAP available). A sequel to the author's 2020 Deep Learning for Vision Systems, focused entirely on CV through the lens of deep learning.

Documentation and Tutorials

OpenCV documentation at opencv.org. Every feature extractor and classifier discussed in this module is implemented in OpenCV with worked examples and a tutorial library.

scikit learn user guide at scikit-learn.org. The standard library for classic ML in Python. The user guide is excellent, free, and example driven.

PyImageSearch tutorials by Adrian Rosebrock at pyimagesearch.com. Practical hands on tutorials covering HOG, SIFT, SVM, and many of the topics in this booklet.

Foundational Papers

Dalal, Navneet and Triggs, Bill. Histograms of Oriented Gradients for Human Detection. IEEE Computer Vision and Pattern Recognition, 2005. The original HOG paper.

Lowe, David. Distinctive Image Features from Scale-Invariant Keypoints. International Journal of Computer Vision, 2004. The original SIFT paper.

Viola, Paul and Jones, Michael. Rapid Object Detection using a Boosted Cascade of Simple Features. IEEE Computer Vision and Pattern Recognition, 2001. The original real time face detection paper.

Cortes, Corinna and Vapnik, Vladimir. Support Vector Networks. Machine Learning, 1995. The foundational SVM paper.

Breiman, Leo. Random Forests. Machine Learning, 2001. The original Random Forest paper.

Looking Ahead to Module 04

You finish this module knowing how computer vision worked for forty years and why it still works today in much of the industry. In Module 04 you will see what happened in 2012 when everything changed.

Module 04 introduces neural networks. Instead of humans designing the feature extractor and the machine learning only the classifier, neural networks learn both at once, end to end, from raw pixels. The shift sounds small. It was not. It moved the accuracy ceiling on standard benchmarks by enough that many problems went from unsolvable to routine in a few years.

You will start with the biological inspiration (real neurons), move to the artificial neuron called the perceptron, stack perceptrons into networks called multi layer perceptrons, and learn how networks train themselves through forward passes, loss functions, gradient descent, and backpropagation. By the end you will be able to draw a simple network, explain how it learns, and know why basic networks struggle with images, which sets up Module 05 on convolutional neural networks.

Everything you learned in this module carries forward. The vocabulary stays. The pipeline structure stays. The notion of training, validation, and test sets stays. The lifecycle stays. What changes is the boundary between human design and machine learning. Watch that boundary move in Module 04.

Questions about this module? email or message me if you have any questions.